Models

Let's think about the database structure first. We need products to sell and transactions to keep a record of the sales:

id	uuid	title	price	url	user_id
1	9b058043-763c-46a3-ad28-33a013f03a1f	Domain-Driven Design with Laravel	3900	https://domain-driven-design-laravel.com/	1
2	9b058046-145d-4a83-aa1e-02f6baad708c	DevOps with Laravel	3900	https://devops-with-laravel.io/	1

`3900` means `3900` cents which is $39. As I said earlier, I'm going to store prices as integers in cent values.

Every table has an `id` and a `uuid` column as well. The reason is this:

- An auto-increment integer value is the best option for MySQL to use in foreign key constraints, `join` statements, or in BTREEs. This is the fastest ID column type by design.
- But I don't want to expose integer IDs in API URLs because it's a security risk. It exposes (at least) two problems
  - The German tank problem. So in WW2 Germans built lots of tanks and the Allies wanted to know how many. Germans used sequential serial numbers such as an auto-increment ID. When the Allies captured a German tank they were able to guess how many did the Germans build in a month based on the serial number. When you expose integer IDs you reveal information about your data. It may or may not be a problem in your specific case. If we expose product IDs we publicly reveal the number of products in the application. Probably not a big problem. But what if we also expose the ID of transactions by a user? For example: `/api/users/1/transactions/9` We just "revealed" some business or revenue-related information about user `1`.
  - Enumeration attack problems. If you make an authorization mistake and you use integer IDs in your API, you're in trouble. Imagine that you forgot to authorize the `DELETE /api/products/{id}` endpoint in your `Request` class. Now anyone can wipe your products table by just running this:

```
#!/bin/bash
for ((i=1; i<=100; i++)); do
  curl -X DELETE "https://myawesomeapp.com/api/products/$i"
done
```

This is why I'm using both integer IDs and UUIDs. Auto-increment IDs are being used internally in database queries, but only UUIDs are exposed via APIs.

Now let's take a look at the `transactions` table: